RISC-V / COMPILER LEARNING PLAN

RISC-V 编译入门 一周学习计划

以 LLVM 为主线 · 从标量代码生成到 RVV 向量化

从“一段 C 代码如何变成 RISC-V 指令”入手，逐步建立指令集、ABI、交叉编译、LLVM 后端与向量化之间的联系。

指令集与 ABI → 交叉编译 → LLVM 后端 → RVV 向量化

投入与预期

按熟悉 C/C++、每天投入 2–3 小时安排。建议每天阅读 45 分钟、实验 60–90 分钟、整理笔记 15 分钟。具体节奏可按已有编译基础调整。

> 一周后：能读懂简单 RISC-V 汇编，跑通 C → IR → 汇编 → 目标文件的流程，找到后端关键实现，并初步判断循环能否向量化。

日期	主题	当日产出
第 1 天	认识 RISC-V 编译目标	三个小函数及汇编对照
第 2 天	工具链与 ABI	函数调用、栈帧和目标文件笔记
第 3 天	LLVM 编译流程	C、LLVM IR、汇编对应关系
第 4 天	阅读 RISC-V 后端	指令与测试的源码定位记录
第 5 天	RVV 基础	标量与向量循环对照
第 6 天	编译器如何使用 RVV	自动向量化与 intrinsics 分析
第 7 天	完成最小案例	代码、命令、测试与一页总结

内容依据：前述学习计划及工作区资料。整理日期：2026-10-19。
文档中的蓝色参考链接可点击；源码路径以当前工作区为准。

DAY 01–02 / ARCHITECTURE & TOOLCHAIN

先读懂编译器面向的机器

第 1 天 · 认识 RISC-V 编译目标

学习重点：RV32/RV64、通用寄存器、整数运算、访存与分支；了解 I/M/A/F/D/C/V 扩展的用途。

1. 写加法、条件判断、数组求和三个 C 函数。
2. 分别用 -O0 与 -O2 生成 RISC-V 汇编，标记优化前后的主要变化。
3. 对照寄存器名称与指令手册，解释参数、返回值和循环控制。

> 验收：能解释 a0、sp、ra；看懂 add/addw、lw/ld、条件跳转和 ret。知道 RV64 中仍会出现处理 32 位数据的指令。

阅读：RISC-V Assembly Programmer's Manual 的寄存器、常见指令和伪指令。实验代码与命令见第 5 页。

第 2 天 · 工具链与 ABI

学习重点：target triple、-march、-mabi、-mcpu、-mtune；参数传递、返回值、栈对齐和寄存器保存；了解 sysroot 的作用。

1. 编写有函数调用的例子，用 noinline 保留调用关系。
2. 观察参数如何传递、何时保存 ra、栈帧如何建立和恢复。
3. 生成 .o 文件并反汇编，了解目标文件和最终可执行文件的区别。

> 验收：能说明 ISA 与 ABI 的区别，以及为什么生成汇编成功不代表一定能链接和运行。

配置项	需要回答的问题
target triple	目标架构、操作系统和环境是什么？
-march / -mabi	允许哪些指令？函数与二进制接口遵循什么约定？
-mcpu / -mtune	针对哪个处理器选择特性或调整优化策略？
sysroot	目标环境的头文件、库等资源从哪里获取？

阅读：RISC-V ELF psABI 的 Integer Calling Convention、寄存器保存规则与栈对齐；LLVM RISC-V User Guide 的目标配置。

DAY 03–04 / LLVM & CODE GENERATION

把 C、IR、指令与后端源码连起来

第 3 天 · LLVM 编译流程

学习重点：Clang、LLVM IR、opt、llc、汇编器与链接器各自的作用；认识 load/store、br、phi、getelementptr。

C → Clang / LLVM IR → 优化 → 后端代码生成 → 目标文件 → 链接

1. 把同一程序保存为 .ll、.s、.o，并记录每条生成命令。
2. 观察常量折叠、无用代码删除和循环优化在 IR 中的表现。
3. 选择一个循环，把 C 中的访存、运算、条件与对应 IR、汇编逐项对应。

> 验收：能说清各阶段输入与输出，能把 C、IR 和汇编中的主要操作对应起来。

第 4 天 · 阅读 RISC-V 后端

学习重点：指令选择、合法化、寄存器分配与栈帧处理；了解 TableGen 如何描述指令及匹配规则。

1. 从简单 add 的 CodeGen 测试入手，先读输入 IR、预期汇编和 RUN 命令。
2. 找到对应指令定义与匹配规则，修改输入 IR，观察生成结果。
3. 按需查阅 lowering、调用约定和栈帧相关实现，画一张源码位置图。

> 验收：知道指令定义、lowering、调用约定和回归测试分别位于哪里。

本地入口	用途
test/CodeGen/RISCV/alu64.ll	最小 IR 与汇编预期
lib/Target/RISCV/RISCVInstrInfo.td	指令定义与匹配模式
lib/Target/RISCV/RISCVISelLowering.cpp	目标相关 lowering
lib/Target/RISCV/RISCVFrameLowering.cpp	栈帧建立与恢复

上述路径均相对于 /root/MIC-for-learning/llvm-project/llvm/。
阅读：LLVM Code Generator 的流程、指令选择和 MachineInstr 部分。

DAY 05–06 / RISC-V VECTOR EXTENSION

理解向量循环为什么这样生成

第 5 天 · RVV 基础

学习重点：VLEN、VL、SEW、LMUL，mask、尾部策略与 vsetvli；认识 LLVM scalable vector。

概念	先掌握的含义
VLEN	一个向量寄存器的位宽，由硬件实现决定
VL	当前一次向量操作采用的元素数量
SEW / LMUL	元素位宽 / 向量寄存器分组倍率
mask / 尾部策略	控制哪些元素参与操作，以及未参与部分如何处理

1. 用支持相应 RVV 功能的编译器编译数组加法，对比 rv64gc 与 rv64gcv。
2. 结合向量化诊断，观察是否生成向量循环、长度设置与尾部处理。
3. 记录编译器版本、目标参数以及关键汇编，避免只凭是否出现向量指令判断优化质量。

> 验收：能解释一次循环处理多少元素，以及 VL 与向量寄存器宽度的区别。

阅读：RISC-V Vector Primer 的长度、元素宽度、寄存器分组和循环处理。

第 6 天 · 编译器如何使用 RVV

学习重点：自动向量化与 C intrinsics；别名、循环依赖、成本模型对向量化的影响。

1. 比较普通循环、添加合法 restrict 的循环、intrinsics 实现。
2. 观察优化诊断、IR 与汇编；intrinsics 只查询设置长度、加载、加法、存储接口。
3. 对未向量化的循环分类：无法证明变换安全、成本不划算，或目标 / 编译器支持不足。

> 验收：能解释三种实现的差异，并基于诊断提出下一步调查方向。

阅读：Vector C Intrinsics、Intrinsics Viewer。进一步查看后端 RISCVInsertVSETVLI.cpp，理解向量配置指令如何插入。

RVV 的 intrinsic 接口与支持情况受编译器版本影响。记录所用版本并匹配文档；已有本地源码和已安装工具可能来自不同版本。

LAB / C → IR → ASSEMBLY → OBJECT

基础实验：数组加法的编译路径

保存为 demo.c。样例不依赖标准库，适合先观察代码生成；restrict 的使用要求调用时满足相应的不别名约束。

```
long add(long a, long b) {
    return a + b;
}

void add_arrays(int *restrict out,
                const int *restrict a,
                const int *restrict b,
                long n) {
    for (long i = 0; i < n; ++i)
        out[i] = a[i] + b[i];
}
```

在样例目录执行以下命令。比较优化级别时，可另存 -O0 版本并与 -O2 结果对照。

```
# C -> LLVM IR
/usr/bin/clang --target=riscv64-unknown-elf \
  -march=rv64gc -mabi=lp64d -O2 \
  -S -emit-llvm demo.c -o demo.ll

# LLVM IR -> RISC-V assembly
/usr/bin/llc -mtriple=riscv64-unknown-elf \
  -mattr=+m,+a,+f,+d,+c -target-abi=lp64d \
  demo.ll -o demo.s

# C -> object file
/usr/bin/clang --target=riscv64-unknown-elf \
  -march=rv64gc -mabi=lp64d -O2 \
  -c demo.c -o demo.o

# Disassemble
/usr/bin/llvm-objdump -d demo.o
```

原计划中的环境检查：系统 Clang/LLVM 为 14，支持 RISC-V；工作区 llvm-project/build/bin/llc 为 LLVM 18，但未构建 RISC-V 后端。以上命令在制定原计划时已验证能生成 IR、汇编与目标文件；本次整理 PDF 未重新运行实验。

前三天可使用系统工具；RVV 部分可先使用 Compiler Explorer 中较新的 RISC-V Clang，并记录版本。本实验尚未链接和运行程序，不代表完成运行时正确性或性能验证。

DAY 07 / REVIEW & REFERENCES

用一个完整小案例结束第一周

第 7 天 · 整理成果与最小回归测试

1. 以数组加法为主例，整理标量与 RVV 代码，保存工具版本和完整命令。
2. 写一个 llc + FileCheck 测试，检查关键的代码生成结果。有运行环境时，补充长度为 0、1 和非整批长度等正确性验证。
3. 写一页分析：代码如何进入后端、向量化发生在哪里、观察到了什么差异、哪些问题还需要确认。

> 最终交付：C 源码、编译命令、IR / 汇编、最小测试、一页分析笔记。能独立复现并解释一个小例子的完整编译路径。

若工作涉及 MLIR，可补充自己的流程图：高层算子如何经过 dialect 转换进入 LLVM IR，再由 RISC-V 后端生成指令。

按任务查阅的参考资料

对应日期	可点击的资料入口	重点
第 1–2 天	RISC-V Assembly Programmer's Manual	寄存器、指令、伪指令
第 2 天	RISC-V ELF psABI	调用约定、寄存器保存、栈
第 2–3 天	LLVM RISC-V User Guide	目标配置与扩展支持
第 3–4 天	LLVM Code Generator	代码生成、指令选择
第 5 天	RISC-V Vector Primer	向量长度、分组与循环
第 6 天	Vector C Intrinsics RVV Intrinsics Viewer	查阅所需接口
实验工具	Compiler Explorer	在线比较代码生成

资源总入口：riscv/learn。在线文档可能随版本更新，具体支持情况以实验所用编译器为准。

复盘时回答这四个问题

① ISA 与 ABI 分别约束什么？ ② 优化发生在哪个阶段？
③ 一个操作如何变成目标指令？ ④ 向量化的主要约束是什么？